

Lecture 7 Static Timing Analysis

STA(Static Timing Analysis)

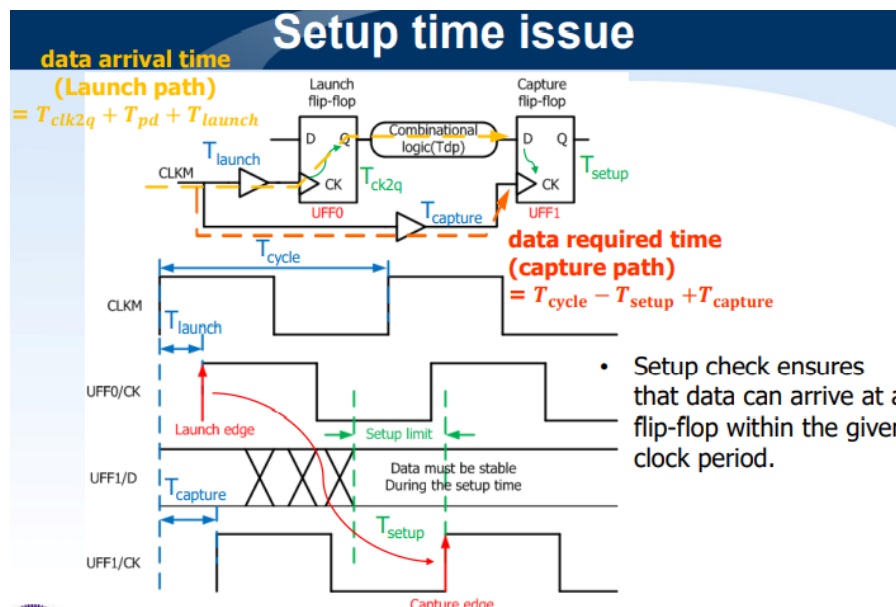
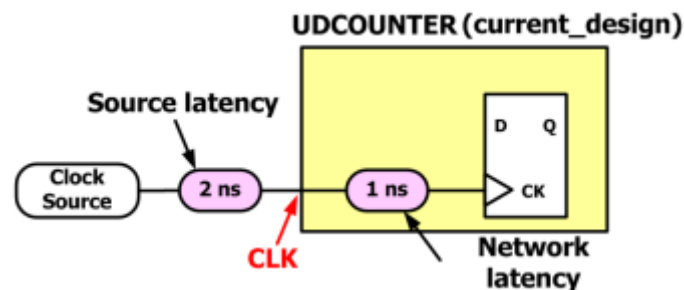
- Verify the timing of a digital design based on a **statistic** method without requiring a simulation of the full circuit
- STA is a **complete** and **exhaustive** verification of all timing checks of a design because it **analyzes every timing paths in design according to constraints.**
- check that if there is any violation on setup/hold time in the timing path, glitches, critical path and clock skew.
- STA aims at different corner to analysis, so the analysis time is faster than DTA.

DTA (Dynamic Timing Analysis)

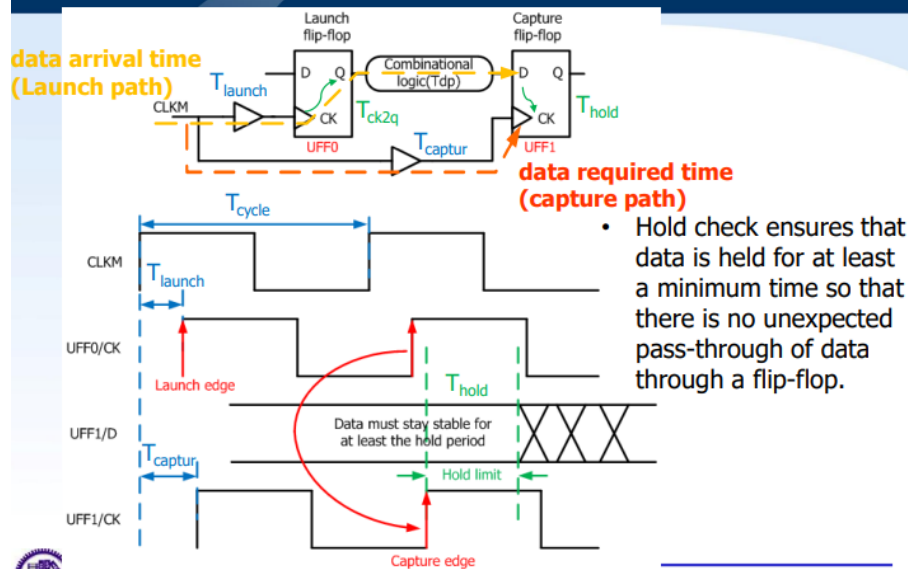
- Quality(Coverage) of the Dynamic Timing Analysis (DTA) increases with the increase of **input test vectors.**
- The growing size of test vectors increase simulation time, especially in post-sim.

Clock source latency: Clock source latency is the delay from the clock source to the clock definition point (CLK); Source latency could represent either on-chip or off-chip latency.

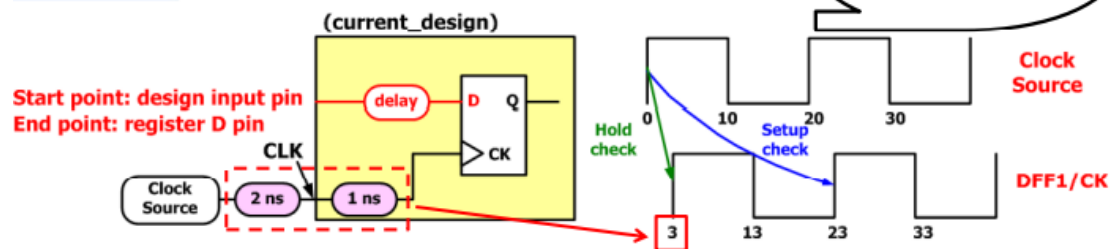
Clock network latency: from the clock definition point (CLK) to the clock pin of a flip-flop.



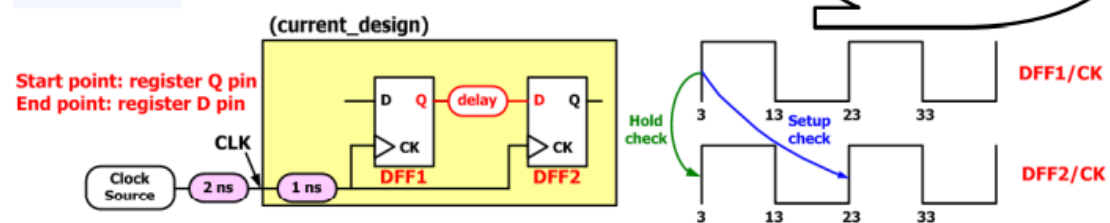
Hold time issue



Timing checks with clock network latency



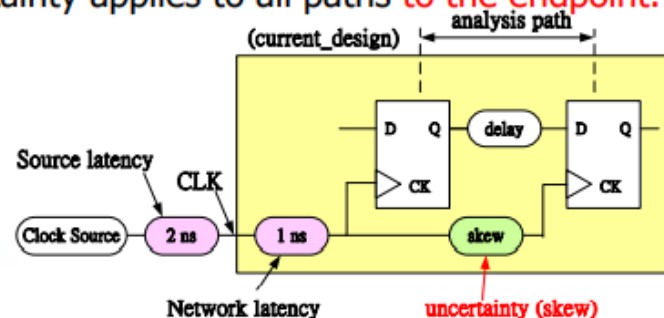
Timing checks with clock network latency



Clock network delay = source latency + network latency

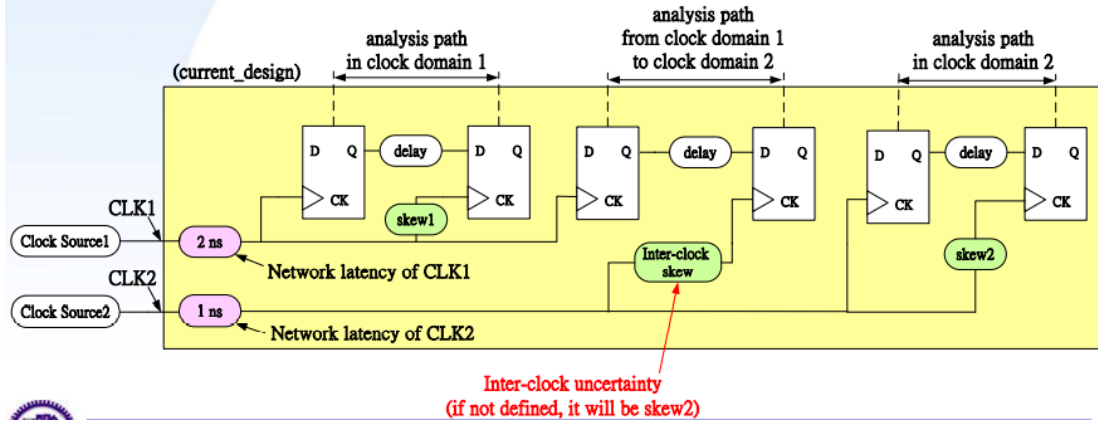
Clock skew of single clock network

- Simple uncertainty** means the setup uncertainty and hold uncertainty applies to all paths to the endpoint.



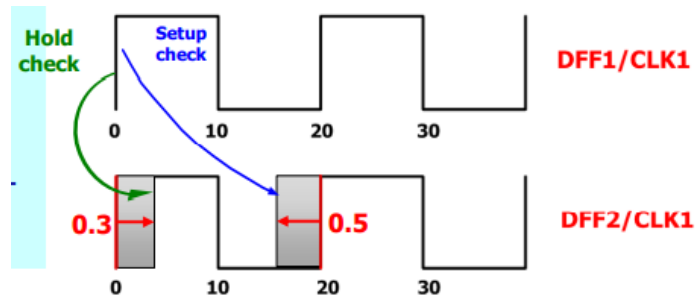
✓ Inter-clock skew of multiple clock networks

- **Inter-clock uncertainty** means the setup uncertainty and hold uncertainty applies to all paths from start point of clock domain1 to the endpoint of clock domain2
- skew1 and skew2 are simple uncertainty of clock domain1 and clock domain 2



Plus uncertainty is for hold time check.

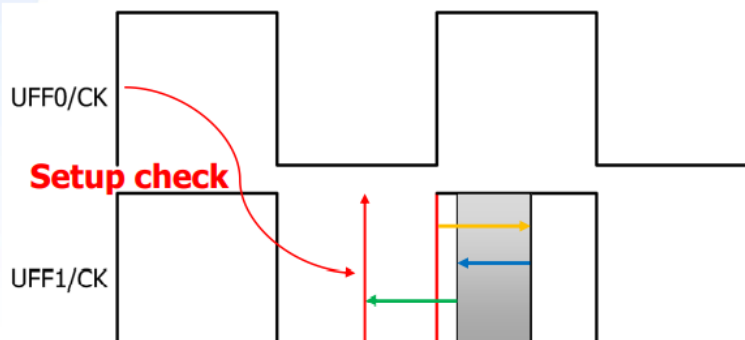
Minus uncertainty is for setup time check.



If inter-clock uncertainty is not defined, compiler will use uncertainty of CLK2 automatically.

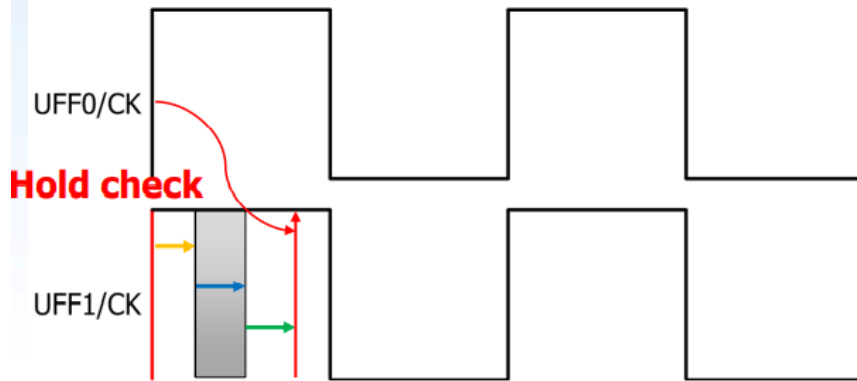
Positive edge trigger circuit timing check

- Setup check
 - check_edge : ideal_clock_rise_edge
 + rise_delay(network latency+source latency)
 – setup_uncertainty(minus uncertainty)
 – T_{setup} (library setup time)



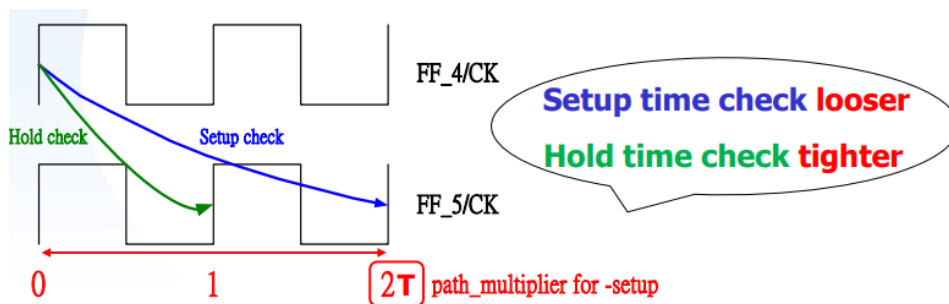
– Hold check

- check_edge : **ideal_clock_rise_edge**
 + rise_delay(network latency+source latency)
 + hold_uncertainty(plus uncertainty)
 + T_{hold} (library hold time)

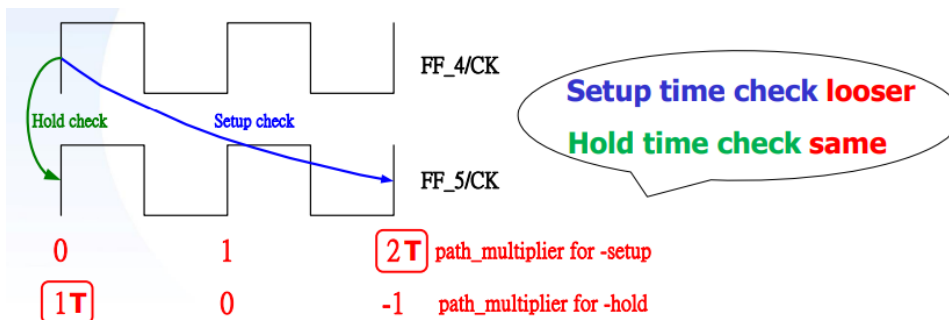


Why multicycle path ? : A timing path which is not expected to propagate a signal in one cycle, set multicycle paths to direct compiler to allow multiple clock cycles for data to propagate along a path.

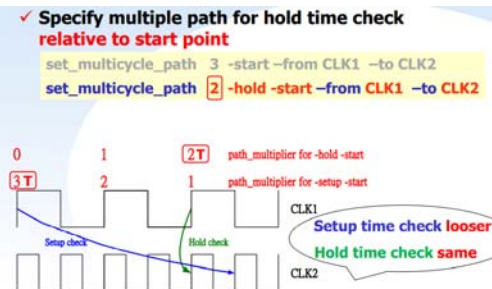
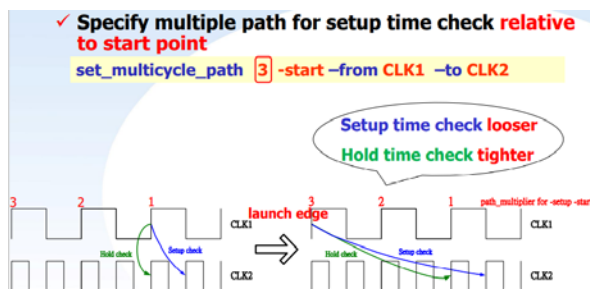
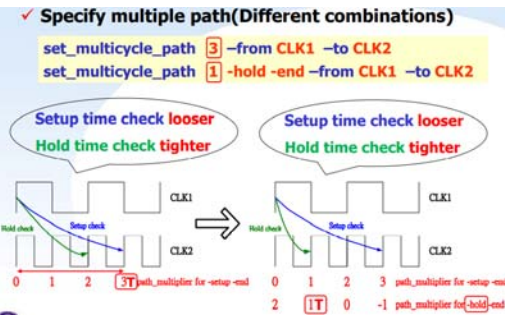
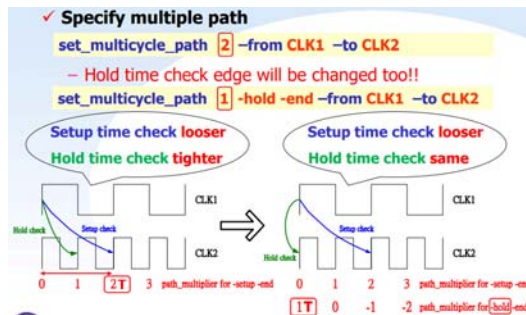
- Specify multiple path for setup time check – Hold time check edge will be changed too!!



- Specify multiple path for hold time check.



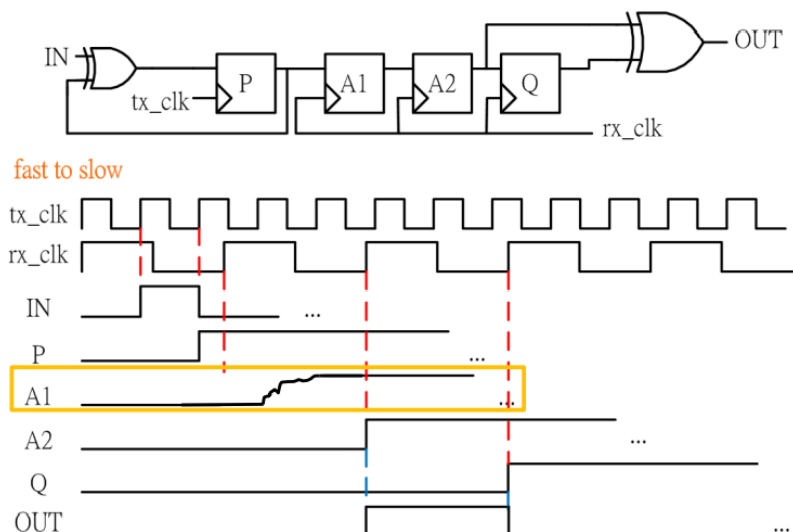
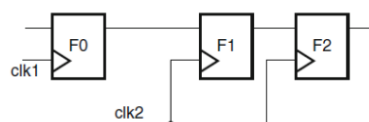
Example:



- Why we need to use **clock domain crossing (CDC)**: Because different blocks require different clock period, some blocks need long clock period to operate, but others only do simple work. If we all use long clock period, it will cause the performance to be poor.
- The unstable status due to non-ideal data transition is called **metastability**.
- CDC solution:

Double Flop(2-FF) Synchronizer

- F1 samples the asynchronous input signal into the destination clock domain, probably a meta-stable value.
- With a full clk2 cycle to permit any meta-stability on the F1 output signal to decay.
- Then the signal is sampled by the same clock into F2, which we expect to be a stable and valid signal.
- Often used in 1-bit flag transmission.



Specify False Path: Remove timing constraints from particular path – Timing checks of false path will be disabled.

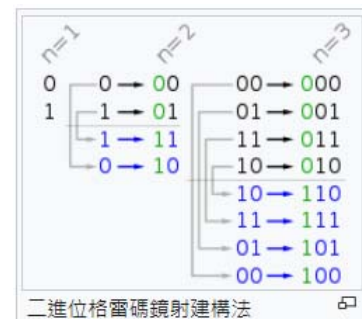
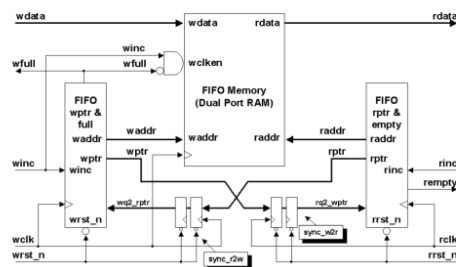
set_annotated_check: Using PrimeTime tool rewrite the sdf file to ignore some timing checks.

Handshake Synchronizers.

- Synchronize a single-bit request into the destination clock domain and wait for a synchronized version of the acknowledge to come back.
- Time uncertainty and high latency.
- Data stability and protocol check

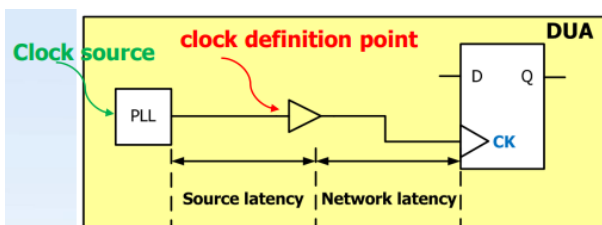
Dual Clock FIFO synchronizers (Asynchronous FIFO)

- Data is written into a dual-port RAM block from the source clock domain and the RAM is read in the destination clock domain.
- Gray-coded read and write pointers are passed into the alternate clock domain to generate full and empty status flags.
- High throughput but difficult hardware.



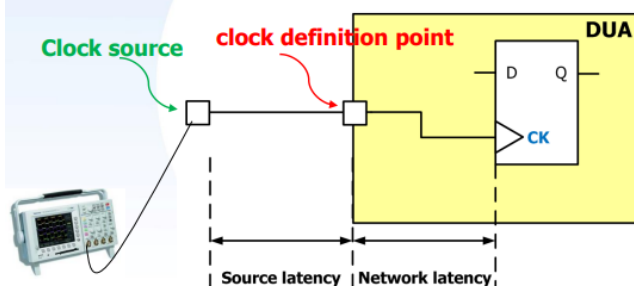
Setup time determines **max** value of tpd.

Hold time determines **min** value of tpd.



p.s. DUA(Design under analysis)

- On-chip clock source

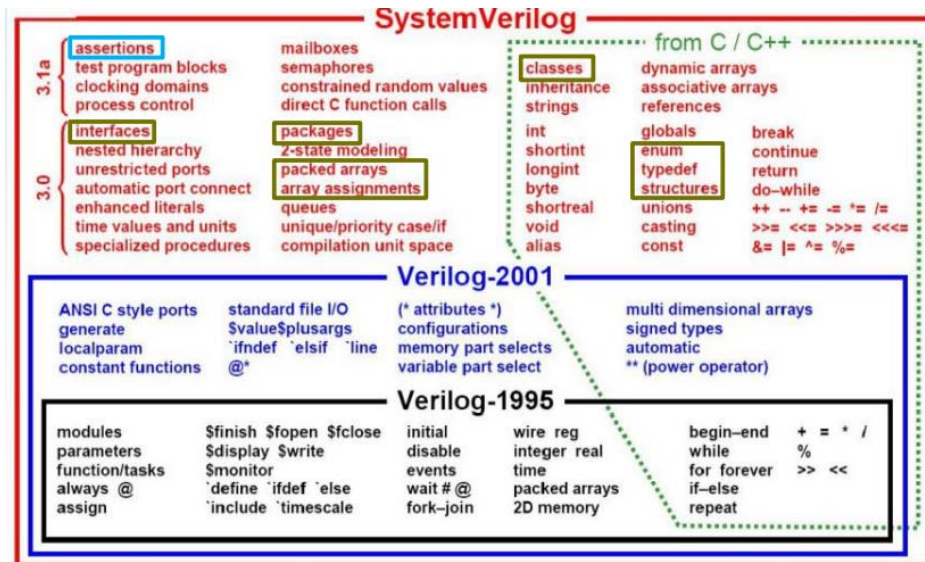
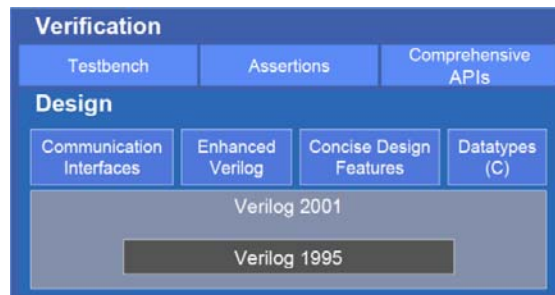


- Off-chip clock source

Lecture 8 SystemVerilog

Why SystemVerilog?

- Designs efficiency– Need to code for reuse and higher abstraction – Need more efficient coding constructs with native language support
- System level hardware design/verification languages
- Testbenches are growing exponentially – Find a way to generate more useful test data and make testing faster



testbed.sv – Connecting testbench and design modules, Generating clock, Dump waveform

design.sv – Design under test (DUT)

pattern.sv – Pattern, Test program, Assertion, Coverage

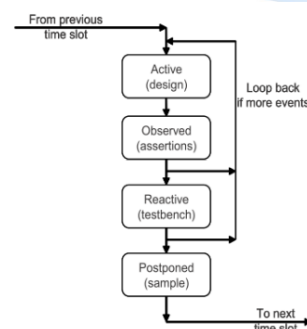
Program:

Purpose: Identifies verification code

A program differs from a module

- Only initial blocks allowed
- Execute in reactive region

```
program name (<port_list>);
  <declarations>; // type, func, class, clocking...
  <continuous_assign>
  initial <statement_block>
endprogram
```



Verilog:

The **Verilog** language uses the **reg** type as a general purpose variable

- Both combinational and sequential circuits can be model using reg
- reg type has no connection to register, which often causes misunderstanding

Lacks of user-defined type

- A dimension declared before the object name is referred to as the "vector width" dimension.
- The dimensions declared after the object name are referred to as the "array" dimensions.

`reg [7:0] r1 [1:256];` // [7:0] is the vector width, [1:256] is the array size

SystemVerilog:

4-State Variables (1|0|Z|X)

(default value is X if not initialized)

- **logic w;** // can be used in both assignment and procedure blocks
 - No input and output restriction anymore
 - No continuous or block procedure restriction
- **reg r;** // the same as Verilog
- **integer i;** // Verilog-2001 >= fixed size (32-bit data type)

```
1 logic net_A, net_B;
2
3 assign net_A = y;
4
5 always @* begin
6     net_B = y;
7 end
```

2-State Variables (1|0)

(default value is 0 if not initialized)

- **shortint s;** // 16-bit signed integer
- **int i;** // 32-bit signed integer
- **longint l;** // 64-bit signed integer
- **byte b8;** // 8-bit signed integer
- **bit b;** // 1-bit integer

"**packed array**" refers to the dimensions declared before the object name (what Verilog-2001 refers to as the vector width)

"**unpacked array**" refers to the dimensions declared after the object name

Example

```
1 bit [3:0] [7:0] test [1:10]; // 10 entries of 4 bytes (packed into 32 bits)
2 // can be used as follows:
3 test [9] = test[8] + 1; // 4 byte add
4 test [7] [3:2] = test [6] [1:0]; // 2 byte copy
```

Typedef:

Make code clear

- SystemVerilog's data type system allows you to define quite complex types. To make this kind of code clear, the **typedef** facility was introduced
- **typedef** allows users to create their own names for type definitions that they will use frequently in their code

Example1:

```
1 typedef reg [7:0] octet;
2 octet b;
3 // is the same as
4 reg [7:0] b;
```

Example2:

```
1 typedef octet [3:0] quadOctet;
2 quadOctet qBytes [1:10];
3 // is the same as
4 reg [3:0] [7:0] qBytes [1:10];
```

Enumerate(Enum):

Create enumerated data types:

- Defines a set of named values, which provides built-in assertion!!
- Data type defaults to **int** (32-bit, 2state, signed int)
- Variable initialized to 0 if initial values aren't specified

Create enum variables:

// anonymous enumerated type

- **enum** <type> {lists_of_enumerations} <enumvar>;
 enum { circle, ellipse, freeform } c;

// typed enumerated type

- **typedef enum** <type> {lists_of_enumerations} <enumtype>; enumtype variable_name;
 typedef enum { circle, ellipse, freeform } ClosedCurve;
 ClosedCurve c;

Often use to represent state machine

Example1:

```
1 // default int type
2 enum {red, yellow, green} light1, light2;
3 enum {bronze=3, silver, gold} medal; // silver=4, gold=5
4 enum {a=0, b=7, c, d=8} alphabet; // Syntax error
5
6 // enumerated type with a 2-bit logic type
7 typedef enum logic [1:0] {WAIT, LOAD, READY} state_t;
8 state_t state, next_state;
9
10 typedef enum logic [2:0]
11 { a=3'd0, b=3'd1, c, d=3'd5 } // ok, c=2
12 { a=3'd0, b=3'd1, c, d=3'd2 } // error!
13 { a=3'd0, b=3'd1, c=3'd7, d } // error!
```

Example2:

```
1 // Must match the size of the base type (default to int)
2 enum logic [2:0] {WAIT=3'b001, LOAD=3'b010, READY=3'b100} state;
3
4 //syntax error!!! (Error enum value size)
5 enum {WAIT=3'b001, LOAD=3'b010, READY=3'b100} state;
```

Structure(Struct):

Group related signals to enhance readability and clearly convey designer's intent (ex: CPU's instruction)

Create structures data types:

- Structures are a collection of variables
 - **typedef struct**{
 <data_type> variable0;
 <data_type> variable1;
}struct_type;

Create structure variables:

- **struct**{
 <data_type> variable0;
 <data_type> variable1;
}struct_variable;

Example:

<pre>1 typedef struct{ 2 int a,b; 3 opcode_t opcode; 4 logic[31:0] address; 5 bit error; 6 }Instruction; 7 8 Instruction IR; 9 IR.address = 32'hF000001E;</pre>	<pre>1 struct{ 2 int a,b; 3 opcode_t opcode; 4 logic[31:0] address; 5 bit error; 6 }IR; 7 8 IR.address = 32'hF000001E;</pre>
---	--

Union:

A SystemVerilog union allows a single piece of storage to be represented different ways using different named member types.

```
typedef logic [3:0] type_A;
typedef logic [7:0] type_B;
typedef type_B [1:0] type_C;

typedef struct packed {
    integer    temp_int; // 32-bits
    type_C     c;         // 16-bits
    logic[15:0] d;         // 16-bits
} type_strct ;           // 64-bits

typedef union packed{
    type_strct mem_strct;
    type_A     [15:0] mem_a;
    type_B     [7:0]  mem_b;
    type_C     [3:0]  mem_c;
}my_union_type;
```

```
my_union_type my_union;
type_A      get_a;
type_B      get_b;
always_ff@(posedge clk or negedge inf.rst_n) begin
    if(!inf.rst_n)begin
        get_a <= 'b0;
    end
    else begin
        get_a <= my_union.mem_a[0];
    end
end
always_ff@(posedge clk or negedge inf.rst_n) begin
    if(!inf.rst_n)begin
        get_b <= 'b0;
    end
    else begin
        get_b <= my_union.mem_b[1];
    end
end
```

Struct vs. Union: <https://gaexp251546.pixnet.net/blog/post/67520022>

Package: To enable sharing a user-defined type definition across multiple modules, SystemVerilog adds packages to the Verilog language.

When the case statement references the enumerated labels of ADD, SUB, and MUL, as well as the function multiplier, it will find the definitions of these names in the **definitions package**

```
module ALU
(input  definitions::instruction_t  IW,
 input  logic                      clock,
 output logic [31:0]              result
);

    always_comb begin
        case (IW.opcode)
            definitions::ADD : result = IW.a + IW.b;
            definitions::SUB : result = IW.a - IW.b;
            definitions::MUL : result = multiplier(IW.a, IW.b);
        endcase
    end
endmodule
```

```
package definitions;
parameter VERSION = "1.1";
typedef enum (ADD, SUB, MUL) opcodes_t;
typedef struct {
    logic [31:0] a, b;
    opcodes_t opcode;
} instruction_t;
endpackage
```

When the case statement references the enumerated labels of ADD, SUB, and MUL, as well as the function multiplier, it will find the definitions of these names in the **definitions package**

```
module ALU
(input  definitions::instruction_t  IW,
 input  logic                      clock,
 output logic [31:0]              result
);
    import definitions::*; // wildcard import

    always_comb begin
        case (IW.opcode)
            ADD : result = IW.a + IW.b;
            SUB : result = IW.a - IW.b;
            MUL : result = multiplier(IW.a, IW.b);
        endcase
    end
endmodule
```

In Verilog, there are 2 kinds of **procedure block** – **initial**, **always**.

System Verilog adds three new logic specific processes to show designers intent:

- always_comb
- always_ff
- always_latch

In program, we may use **forever@** to replace always blocks

```
initial begin
    forever@ (posedge clk)begin
        // do something
    end
end
```

Verilog2001

```
always@( b or c)
a=b&c;
```



SystemVerilog

```
always_comb
a=b&c;
```

The **always_ff** procedural block is used to indicate that the intent to model synthesizable sequential logic behavior.

The **always_latch** procedural block is used to indicate that the intent to model combinational logic with latch in it.

An **interface** is a named bundle of wires, similar to a struct, except that an interface is allowed as a module port, while a struct is not.

```

3 module moduleA ( input bit      clk
4                 , input logic   ack
5                 , input logic   ready
6
7                 , output logic   send
8                 , output logic [31:0] data
9                 );
10    ... // actual module definition here
11 endmodule
12
13 module moduleB ( input bit      clk
14                 , input logic   send
15                 , input logic [31:0] data
16
17                 , output logic   ack
18                 , output logic   ready
19                 );
20    ... // actual module definition here
21 endmodule
22
23 module top;
24    ...
25    clockgen CLOCKGEN (clk); // the clock generator
26
27    moduleA AA (clk, ack, ready, send, data);
28    moduleB BB (clk, send, data, ack, ready);
29 endmodule
30
31
32
33
34

```

```

3 interface intf_AB;
4     logic      ack;
5     logic      ready;
6     logic      send;
7     logic [31:0] data;
8     ... // actual interface definition here
9 endinterface
10
11 module moduleA ( input bit clk
12                 , intf_AB intf1
13                 );
14    ... // actual module definition here
15 endmodule
16
17 module moduleB ( input bit clk
18                 , intf_AB intf2
19                 );
20    ... // actual module definition here
21 endmodule
22
23 module top;
24    ...
25    intf_AB intf(); // the interface declaration
26    clockgen CLOCKGEN (clk); // the clock generator
27
28    moduleA AA ( .clk (clk )
29                , .intf1 (intf)
30                );
31    moduleB BB ( .clk (clk )
32                , .intf2 (intf)
33                );
34 endmodule

```

Modports are used to define direction of signal inside interface.

```

22 interface INF(input bit clk); 40
23     logic reset;              41
24     instr_t IW;               42
25     data_t OUT;               43
26
27     modport PATTERN(          44
28         output reset,         45
29         output IW,            46
30         input OUT             47
31     );                        48
32
33     modport DESIGN(           49
34         input reset,          50
35         input IW,             51
36         output OUT            52
37     );                        53
38
39 endinterface                  54
40

```

```

41 reg SystemClock;
42 INF inf(SystemClock);
43 PATTERN test_p(
44     .clk (SystemClock),
45     .inf (inf.PATTERN)
46 );
47
48 `ifdef RTL
49 ALU dut(
50     .clk (SystemClock),
51     .inf (inf.DESIGN)
52 );
53 `endif

```

```

23 module ALU(
24     input clk,
25     INF.DESIGN inf
26 );
27 // input instr_t IW, output data_t OUT, input clk, reset);
28 // shortint temp;
29 logic [16:0] temp;
30 // logic [16:0] temp_ttt;
31 logic temp_ovf;
32 // Output sequential logic
33 always_ff @(posedge clk)
34 begin : p_OUTPUT_REG_LOGIC
35     if(inf.reset)
36     begin
37         inf.OUT.result<='0;
38         inf.OUT.ovf<='0;
39     end
40     else
41     begin
42         inf.OUT.result<=temp[15:0];
43         inf.OUT.ovf<=temp[16];
44     end
45 end : p_OUTPUT_REG_LOGIC

```

INF.sv TESTBED.sv DESIGN.sv

Object-oriented programming:

Objects, instances of classes

Contents of “objects”

- Data field
- Constructor
- Methods

Each object should do what it should do, rather than several roles.

Interaction between object and the object should be reduced as much as possible.


```

1 class Transcation
2     //number of objects created
3     static int count = 0;
4     //unique instance ID
5     int id;
6     logic [31:0] addr, crc, data[8];
7
8     function new();
9         id = count++;
10    endfunction
11
12    function void display_id;
13        $display("%d", id);
14    endfunction
15
16 endclass

```

```

1 program test;
2
3     /* other codes ...*/
4
5     Transcation t;
6     initial begin
7         repeat(n)begin
8             t = new();
9             t.addr = $random();
10            transmit(t); // user defined task
11        end
12    end
13
14    /* other codes ...*/
15
16 endprogram

```

Randomization:

Two types of random properties are supported:

- rand
- randc

rand properties can assume any legal value:

- Values can repeat without exhausting all possible values

randc properties can be up to 16 bits:

- Exhaust all values before repeating any individual value
- For non-repeating bit values > 16 bits, use concatenation

rand and randc properties are randomized when the class method **randomize()** is called:

- randomize() is built-in with every class
- 1 is returned if successful, 0 if randomization failed

```

class random_interval;
    rand int interval;
    function new ( int seed );
        this.srand(seed);
    endfunction
    constraint limit
    {
        interval inside{1, 2, 3};
    }
endclass

random_interval interval_rand = new(1);
int i;

initial
begin
    i = interval_rand.randomize();
    if(i == 0) $display("ERROR");
end

```

Constraints:

Class properties are constrained in a **constraint block**

- Use **operator** or **distribution** constraints
- Arrays can be constraint with functions like size()

For distribution specification:

- Distributed over a specified range with keyword **inside**:

```

class member
{
    constraint Limit1 {
        sa inside { 5:7, 10, 15 };
        // 5,6,7,10,15 equally weighted probability
    }
}

```

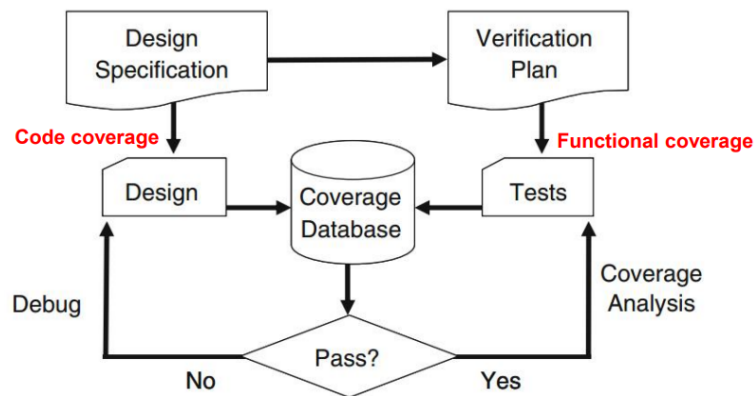
- Excluded from a specified range with **!** operator:

```

constraint Limit2 {
    ! (sa inside { 1:10, 15 });
    // not 1 through 10 or 15
}

```

Lecture 9 SystemVerilog Verification



Multiple cover points sample at the same time are placed together in a cover group. A cover group can sample any visible variable directly such as program variables, signals from an interface, or any signal in the design.

Create a cover group which encapsulates:

- Group of cover points
- Bins definitions
- Coverage goal
- Defining Coverage bins sample timing
- Track progress

```

22 covergroup cg; //start
23 //...
24 //...
25 //...
26 endgroup //end
27
28 cg cg_inst = new(); //instance
  
```

Bins are updated synchronously as the sample_event occurs – Can also use `cov_object.sample()` to update the bins.

```

1 covergroup group_name (argument_list)@(sample_event);
2     coverpoint cp1;
3     coverpoint cp2 {...}
4 endgroup
  
```

IFF:

Event control

- Only be triggered when the expression after iff is true

```

1 @(posedge clk iff(valid));
2 //do_something;
  
```

Good for sampling

```

1 //Example 1
2 covergroup cg1@(posedge clk);
3     coverpoint cp_varib iff(!reset);
4 endgroup
5
6 //Example 2
7 covergroup cg2@(posedge clk iff(!reset));
8     coverpoint cp_varib;
9 endgroup
  
```

SystemVerilog automatically creates a number of bins for cover point.

By default, NC-Verilog automatically creates at most 64 bins.

- Values are equally distributed in each bin
 - 3-bit variable → 8 possible values → 8 autobins will be created
 - 16-bit variable → 65536 possible values → each bins covers 1024 values
- Option **auto_bin_max** specifies the maximum number of bins to create.
 - {option.auto_bin_max = yourdef }
 - 16-bit variable → each bins covers 2048 values

What is **bin**? bin is a container for each value in the given range of possible values of a coverage point variable.

Without auto binning:

- Coverage is :

$$\frac{\text{\# of bins covered (have at_least hits)}}{\text{\# of total bins}}$$

With auto binning:

(auto_bin_max limit the number of bins used in the coverage calculation)

- Coverage is :

$$\frac{\text{\# of bins covered (have at_least hits)}}{\min(\text{possible values for data type} \mid \text{auto_bin_max})}$$

Each covergroup contributes equally.

Within covergroup, each coverpoint/cross block contributes equally.

Define state bins using a range

Define transitions bins using state transitions

```

1 covergroup MyCov @(cov_event);
2   coverpoint port_number{
3
4       bins s0      = {[0: 7]};           //creates 1 state bin
5       bins s1 []   = {[8:15]};           //creates 8 state bins
6                                           //a bin array s1[8] ~ s1[15]
7       ignore_bins ignore = {16,20};      //ignore if hit
8       illegal_bins bad   = default;       //error message if hit
9
10      bins t0       = (0=>8, 9=>0);        //creates 1 transition bin
11      bins t1 []    = ([0:8]>=>[8:15]);    //creates 72 transition bins
12      bins other_trans = default;         //all other transitions
13
14  }
15 endgroup

```

SystemVerilog can automatically create cross coverage bins.

```

covergroup cov1@(posedge clk)
  coverpoint opa;
  coverpoint opb;
  cross opa, opb;
endgroup

```

如果定义两个coverpoint a,b, 如果cross a和b,假定a中有2个bin, b中有2个bin, cross之后一定只有4个bin吗?
不一定。如果a中的bin没有将所有的值涵盖完, 那么系统会分配默认的bin, 那么交叉出来就不止4个bin。

Cross bins for all combinations of the individual state.

at_least(1): – Minimum number of times for a bin to be hit to be considered covered.

auto_bin_max(64): – Maximum number of bins that can be created automatically.

– Each bin contains equal number of values.

per_instance(0): – Keeps track of coverage for each instance when it is set true.

Options control the behavior of the cover group, coverpoint, and cross.

```
7  covergroup address_cov () @ (posedge ce);
8      option.name          = "address_cov";
9      option.comment        = "This is cool";
10     option.per_instance   = 1;
11     option.goal            = 100;
12     ADDRESS : coverpoint addr {
13         option.auto_bin_max = 100;
14     }
15     ADDRESS2 : coverpoint addr2 {
16         option.auto_bin_max = 10;
17     }
18 endgroup
```

Assertion: An assertion is a design condition that you want to make sure never violates.

SystemVerilog assertions have several advantages

- Concise syntax
- Ignore by synthesis
- Can be disabled
- Can have severity level

- `$fatal [ ( finish_number, "message", message_arguments ) ] ;`
 - Terminates execution of the tool
 - `finish_number` is 0, 1 or 2, and controls the information printed by the tool upon exit (the same levels as with \$finish)
- `$error [ ( "message", message_arguments ) ] ;`
 - A run-time error severity; software continues execution
- `$warning [ ( "message", message_arguments ) ] ;`
 - A run-time warning; software continues execution
- `$info [ ( "message", message_arguments ) ] ;`
 - No severity; just print the message

Software tools may provide options to suppress errors or warnings or both

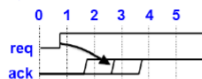
Immediate assertions test for a condition at the current time

```
always @(state)
    assert ($onehot(state)) else $fatal;
```

generate a fatal error state variable is not a one-hot value

An immediate assertion is the same as an if...else statement, but with assertion controls

Concurrent assertions test for a sequence of events over multiple clock cycles



a complex sequence can be defined in very concise code

```
a_reqack: assert property (@(posedge clk) req ##[1:3] ack;) else $error;
```

One line of SVA code replaces all the Verilog code in the example three slides back!

Immediate Assertions:

✓ A test of an expression when the moment the statement is executing

[name:] assert (**expression**) [pass_statement] [else fail_statement]

- May be used in initial, always, tasks, and functions
- Performs a Boolean true/false test
- Evaluates the test at the instant the assert statement is executed

```

always @(negedge reset)
  a_fsm_reset: assert (state == LOAD)
    $display("FSM reset in %m passed");
  else
    $display("FSM reset in %m failed");

```

The name is optional:

- Creates a named hierarchy scope that can be displayed with %m
- Provides a way to turn off specific assertions

Concurrent Assertions:

✓ Test for a sequence of events spread over multiple clock cycles

[name:] assert property (property_spec) [pass_statement] else [fail_statement]

- The property_spec describes a sequence of events
- May be used in initial, always, or stand-alone

```

always @(posedge clock)
  if (State == FETCH)
    ap_req_gnt: assert property (p_req_gnt) passed_count++; else $fatal;
property p_req_gnt:
  @(posedge clock) request ##3 grant ##1 !request ##1 !grant;
endproperty: p_req_gnt

```

request must be true immediately, grant must be true 3 clocks cycles later, followed by request being false, and then grant being false

The argument to assert property() is a **property spec**

- Contains the definition of a sequence of events

```

ap_Req2E: assert property (pReq2E) else $error;
property (pReq2E);
  @(posedge clock) (request ##3 grant ##1 (qABC and qDE));
endproperty: pReq2E

```

A property can reference and perform operations on named sequences

represents a “cycle delay” – Specifies the number of clock cycles to wait until the next expression in the sequence is evaluated.

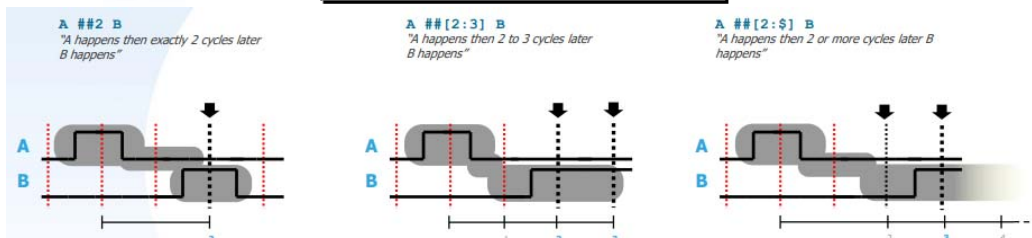
“(posedge clock)” is not a delay, it specifies what ## represents

[min_count:max_count] specifies a range of clock cycles, also they must be non-negative constants

request ##[1:3] grant;

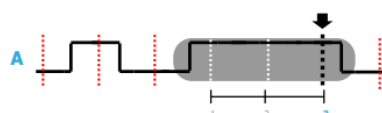
After evaluating request, grant must be true between 1 and 3 clocks later

This sequence would evaluate as true for:
 (request ##1 grant);
 or (request ##2 grant);
 or (request ##3 grant);



Repetition operator [*N] is also sometimes useful:

A [*3]
 "A happens 3 times in a row"



Overlapped $\rightarrow$

- $S1 \rightarrow S2$, If the sequence S1 matches, then sequence S2 must also matches at the same cycle

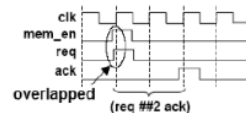
Non-overlapped $\Rightarrow$

- $S1 \Rightarrow S2$, If the sequence S1 matches, then at the next cycle, sequence S2 must also matches

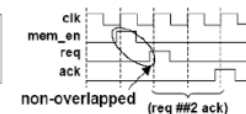
Preconditioned with an implication operator

- If the condition is true, sequence evaluation starts immediately ($\rightarrow$) or next cycle ($\Rightarrow$), otherwise it acts as if it succeeded

```
property p_req_ack;
  @(posedge clk) mem_en  $\rightarrow$  (req ##2 ack);
endproperty: p_req_ack
```



```
property p_req_ack;
  @(posedge clk) mem_en  $\Rightarrow$  (req ##2 ack);
endproperty: p_req_ack
```



IC

and

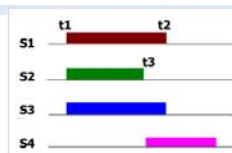
- s1 and s2, succeeds if s1 and s2 succeed. The end time is the end of the sequence that terminates last

intersect

- s1 intersect s3, succeeds if s1 and s3 succeed and if end time of s1 is the same with the end of s3

Or

- s1 or s4, succeeds whenever at least one of two operands s1 and s4 is evaluated to true



'intersect'

Seq1 and Seq2 must START at the same time.

A match occurs only when both sequences end at the same time. If Seq2 ends at a different time, there won't be a match

'and'

Seq1 and Seq2 must start at the same time.

A match occurs whenever the longer sequence ends.

\$rose

- asserts that if the variable changes from 0 to 1 between one posedge clock and the next, detect must be 1 on the following clock.

```
assert property
  (@(posedge clk) $rose(in)  $\Rightarrow$  detect);
```

\$fell

- asserts that if the variable changes from 1 to 0 between one posedge clock and the next, detect must be 1 on the following clock

```
assert property
  (@(posedge clk) $fell(in)  $\Rightarrow$  detect);
```

✓ \$stable

- states that data shouldn't change while enable is 0.

```
assert property
  (@(posedge clk) enable == 0  $\Rightarrow$  $stable(data));
```

\$past

- provides the value of the signal from the previous clock cycle

```
$past(signal_name, number of clock cycles)
```

```
property p;
  @(posedge clk) b  $\rightarrow$  ($past(a,2) == 1);
endproperty
```


Lecture 10 Low Power Design

Why low power? -- Power↑ → Temperature↑ → Safety↓

Power Dissipation:

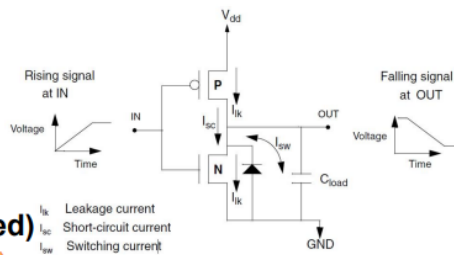
$$P_{\text{total}} = P_{\text{static}} + P_{\text{dynamic}}$$

Static power (P_{static})

- $P_{\text{static}} = I_{\text{leakage}} * V_{\text{dd}}$
 - I_{leakage} : leakage current

Dynamic power (dominated)

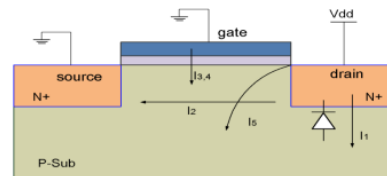
- $P_{\text{dynamic}} = pt * (P_{\text{sw}} + P_{\text{sc}})$
 - pt: Switching Probability of one clock cycle
- **Switching power (P_{sw})**
 - Charging and discharging parasitic capacitance
- **Short circuit power (P_{sc})**
 - Direct path between Vdd and GND when switching



Static Power Dissipation:

Caused by leakage current

- **Sub-threshold current**
 - Dominate the leakage current
 - Increase with temperature
- **Reverse leakage current**
 - Happen when device is reverse biased
 - Increase with temperature
- **Gate Leakage current**
 - Increase with Vdd



I1-Reverse Bias p-n Junction Leakage
I2-Sub Threshold/ Weak Inversion Current
I3-Gate Leakage, Tunneling Current through Oxide
I4-Gate Current due to hot carrier Injection
I5-Gate Induced Drain Leakage(GIDL)

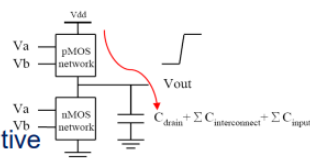
How to minimize static power?

- Reduce voltage supply (**Process dependent**)
- In general: the leakage current can't be changed if process and cells are decided

Dynamic Power Dissipation:

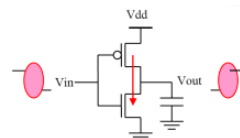
$$P_{\text{dynamic}} = pt * (P_{\text{sw}} + P_{\text{sc}})$$

- **pt : Switching Probability**
- $P_{\text{sw}} = C_L * V_{\text{dd}}^2 * f_p$
 - Charge and discharge the loading capacitive
- **Psc: Short circuit power**
 - Both NMOS and PMOS are turned on

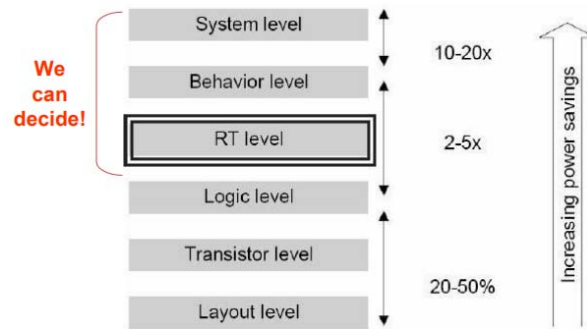


How to minimize dynamic power?

- Reduce Vdd (**process dependent**) → P_{sw} & P_{sc}
- Reduce parasitic capacitance → P_{sw}
- Reduce the overlap time of PMOS and NMOS turn-on time, i.e., keep the input signal rise/fall time the same → P_{sc}
- Reduce unnecessary switching activities → P_t
 - Gated clock (Turn off unused circuits)
 - Register retiming



Where to reduce power?



Static Power Reduction -- Multi Threshold Voltage

$V_t \downarrow$, $I_{sub} \uparrow$, $I_{ds} \uparrow$, speed $\uparrow$, delay $\downarrow$

As V_t decrease, sub-threshold leakage increases -- $V_t \downarrow$, $I_{DS_sub} \uparrow$, $I_{D(SAT)} \uparrow$

When Power is a concern : High V_t logic gate

When Delay time is a concern : Low V_t logic gate

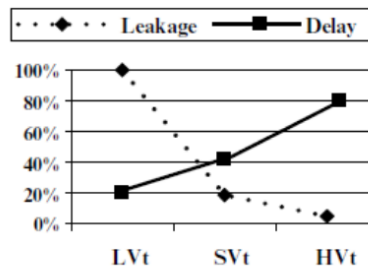


Figure 2-5 Leakage vs. Delay for a 90nm Library

Dynamic Power:

$$P_{dyn} = \underbrace{(C_{eff} \cdot V_{dd}^2 \cdot f_{clock})}_{\text{Switching power}} + \underbrace{(t_{sc} \cdot V_{dd} \cdot I_{peak} \cdot f_{clock})}_{\text{Short circuit power}}$$

Reducing the V_{dd} is the most straight forward way to reduce the power. However, decreasing V_{dd} would cause the circuit delay increasing.

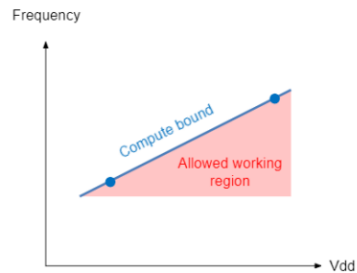
Multi Voltage Strategies:

- **Static Voltage Scaling (SVS):** - Different blocks or subsystems are given different, fixed supply voltages.
- **Multi-level Voltage Scaling (MVS):** - A block or subsystem is switched between two or more voltage levels. - Only a few, fixed, discrete levels are supported for different operating modes.
- **Dynamic Voltage and Frequency Scaling (DVFS):** - An extension of MVS where a larger number of voltage levels are dynamically switched to follow changing workloads.

Dynamic Voltage and Frequency Scaling (DVFS):

- Compute bound in DVFS

- Chips can only work at the region below compute bound.
- DVFS must adjust vdd first before raising frequency, and drop frequency first before turning down vdd.



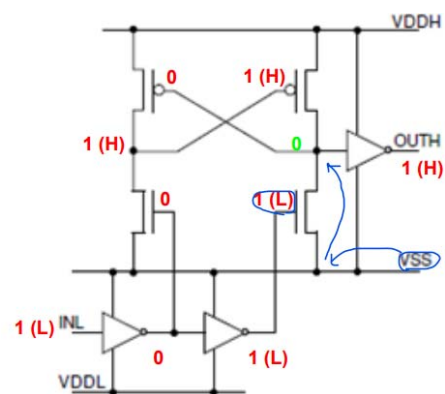
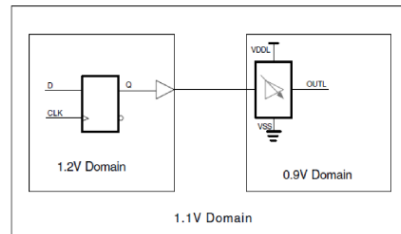
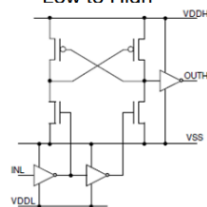
Level Shifter:

Connecting 2 power domains at different voltage levels can cause design issues

- Timing inaccuracy
- Signals are not propagated

A level shifter is required

- High to low
- Low to High



Power Gating:

- The basic idea of power gating is to provide two power modes : A **low power mode** and an **active mode**.
- The goal is to switch between these mode to maximize power savings while minimizing the impact to performance.
- Sleep mode (low power mode) : shut down power to block of logic.

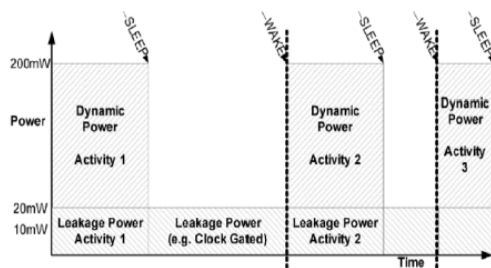


Figure 4-1 Activity Profile with No Power Gating

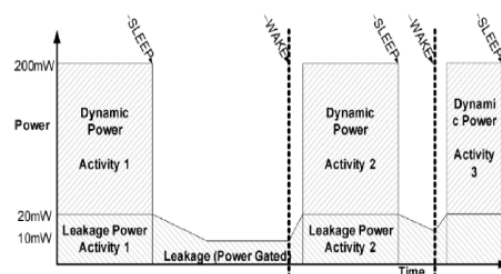
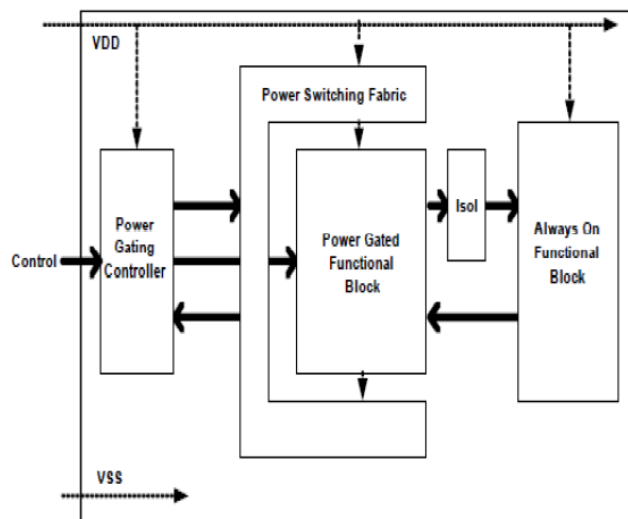
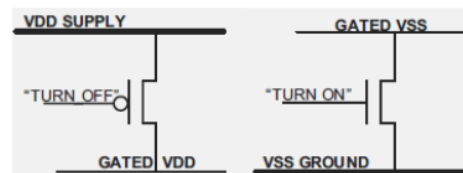


Figure 4-3 Realistic Profile with Power Gating



Power switching fabric :



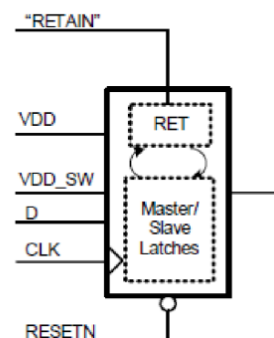
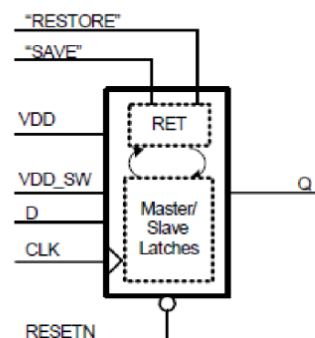
Example: (Isolation)

"X" 0 "ISOLN" 0 CLAMPED (LOW) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>		0	1	0	0	0	1	0	1	"X" 1 "ISOL" 1 CLAMPED (HIGH) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>		0	1	0	0	1	1	1	1	"A" 1 "ISOLN" 1 CLAMPED (LOW) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>		0	1	0	0	0	1	0	1	"X" 1 "ISOL" 1 CLAMPED (HIGH) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>		0	1	0	0	1	1	1	1
	0	1																																					
0	0	0																																					
1	0	1																																					
	0	1																																					
0	0	1																																					
1	1	1																																					
	0	1																																					
0	0	0																																					
1	0	1																																					
	0	1																																					
0	0	1																																					
1	1	1																																					
"X" 1 "ISOLN" 1 CLAMPED (LOW) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>		0	1	0	0	0	1	0	1	"X" 1 "ISOL" 1 CLAMPED (HIGH) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>		0	1	0	0	1	1	1	1	"X" 0 "ISOLN" 0 CLAMPED (LOW) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>		0	1	0	0	0	1	0	1	"A" 0 "ISOL" 0 CLAMPED (HIGH) ISOLATED SIGNAL <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>		0	1	0	0	1	1	1	1
	0	1																																					
0	0	0																																					
1	0	1																																					
	0	1																																					
0	0	1																																					
1	1	1																																					
	0	1																																					
0	0	0																																					
1	0	1																																					
	0	1																																					
0	0	1																																					
1	1	1																																					

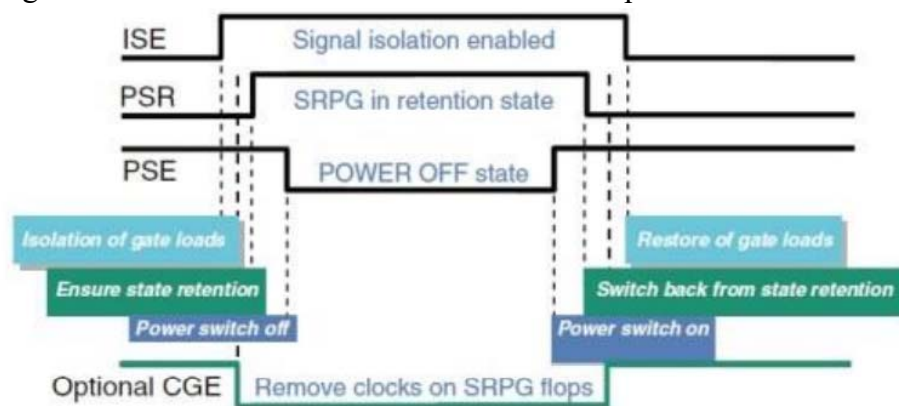
Retention(保留) registers:

States of some registers in shut-down mode need to be preserved

Retention registers can **store data while in shut-down mode**



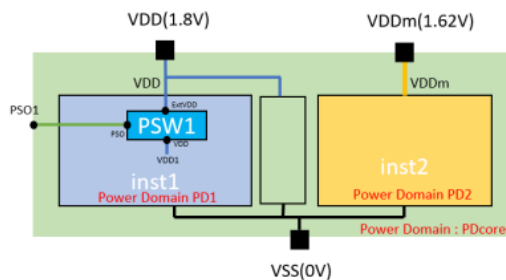
Power gating needs a control circuit to schedule the whole procedure.



*SRPG : State Retention Power Gating Cell

Unified Power Format (UPF):

- Unified Power Format: TCL-based power specification file.
- UPF files consists of commands and objects that describe power intent.



- Create supply nets & supply ports
- Create supply sets and associate power domains
- Create power switches
- Set isolation strategies
- Set level shifter strategies
- Create power state table

Architecture Trade-offs: Power/Frequency/Area

- Pipeline
- Parallel

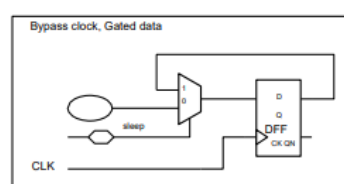
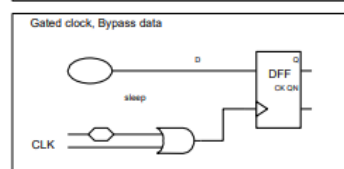
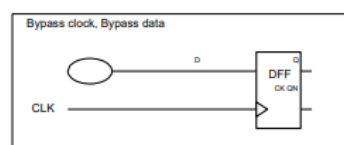
Dynamic Power Gating:

Why do we proceed clock gating

- Reduce the power consumption of registers by **turn off the un-used registers**
- Reduce the clock switching power

Methods

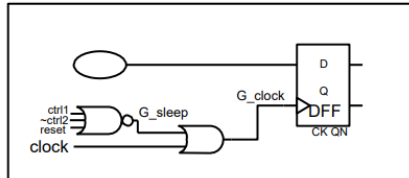
- Gated clock, Bypass data.
- Bypass clock, Gated data.



Gated clock, bypass data

- For a design on ASIC, we do clock gating to reduce the power consumption.
- Notice that the gated condition should be chosen carefully

Gated clock, Bypass data



```

`ifdef SUPPORT_POWER_DOWN
wire G_clock;
wire G_sleep = ~( reset | ctrl1 | (~ctrl2) );
GATED_OR GATED_G (
    .CLOCK( clock ),
    .SLEEP_CTRL( G_sleep ), // gated clock
    .CLOCK_GATED( G_clock )
);
`else
wire G_clock = clock;
`endif

always@(posedge G_clock) begin
    // bypass data
    if (reset) begin
        ..... end
    end else if (ctrl1) begin
        ..... end
    end else if (~ctrl2) begin
        ..... end
end

```

```

`ifdef SUPPORT_POWER_DOWN
wire G_clock;
wire G_sleep = ~( reset | ctrl1 | (~ctrl2) );
GATED_OR GATED_G (
    .CLOCK( clock ),
    .SLEEP_CTRL( G_sleep ), // gated clock
    .CLOCK_GATED( G_clock )
);
`else
wire G_clock = clock;
`endif

always@(posedge G_clock) begin
    // bypass data
    if (reset) begin
        ..... end
    end else if (ctrl1) begin
        ..... end
    end else if (~ctrl2) begin
        ..... end
end

```

G_sleep = 1
G_clock = 1
Clock gated !!

```

`ifdef SUPPORT_POWER_DOWN
wire G_clock;
wire G_sleep = ~( reset | ctrl1 | (~ctrl2) );
GATED_OR GATED_G (
    .CLOCK( clock ),
    .SLEEP_CTRL( G_sleep ), // gated clock
    .CLOCK_GATED( G_clock )
);
`else
wire G_clock = clock;
`endif

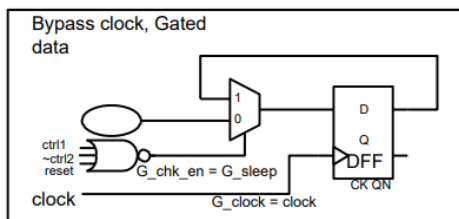
always@(posedge G_clock) begin
    // bypass data
    if (reset) begin
        ..... end
    end else if (ctrl1) begin
        ..... end
    end else if (~ctrl2) begin
        ..... end
end

```

G_sleep = 0
G_clock = clock
Clock bypass !!

Bypass clock, Gated data

- For a design on FPGA, the clock tree is pre-generated. **It is difficult to implement dynamic gated clock scheme.** Thus we do **data gating to minimize signal switching.**
- In order to logic equivalent with dynamic gated clock design, the data gating condition should be same with dynamic gated clock condition.



```

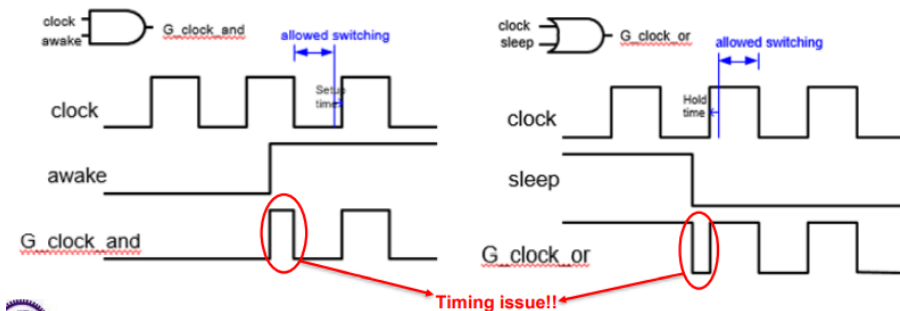
`ifdef SUPPORT_POWER_DOWN
wire G_clock;
wire G_sleep = ~( reset | ctrl1 | (~ctrl2) );
);
wire G_chk_en = G_sleep;
GATED_OR GATED_G (
    .CLOCK( clock ),
    .SLEEP_CTRL( 1'b0 ) // bypass clock
    .CLOCK_GATED( G_clock )
);
`else
wire G_clock = clock;
`endif

always@(posedge G_clock) begin // gated data
    if ( !G_chk_en ) begin
        if (reset) begin
            ..... end
        end else if (ctrl1) begin
            ..... end
        end else if (~ctrl2) begin
            ..... end
        end
    end
end

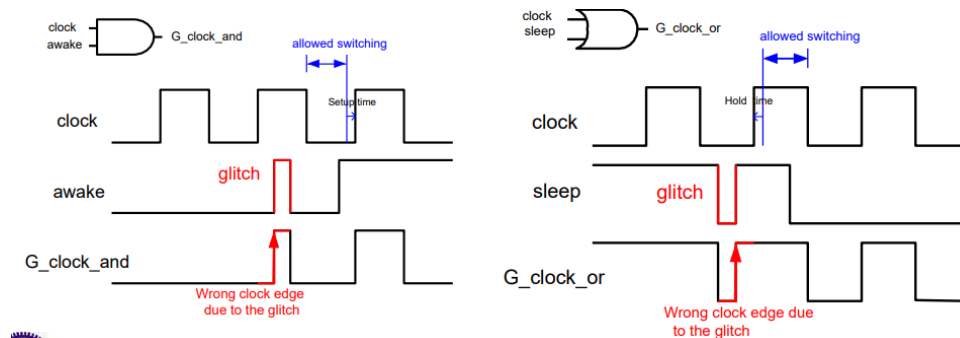
```


<pre> `ifndef SUPPORT_POWER_DOWN wire G_clock; wire G_sleep = ~( reset   ctrl1   (~ctrl2) ); wire G_chk_en = G_sleep; GATED_OR GATED_G ( .CLOCK( clock ), .SLEEP_CTRL( 1'b0 ),           // bypass clock .CLOCK_GATED(G_clock) ); `else wire G_clock = clock; G_sleep = 1 G_chk_en = 1 `endif Data gated !! always@(posedge G_clock) begin // gated data if (!G_chk_en) begin if (reset) begin end else if (ctrl1) begin end else if (~ctrl2) begin end end end end </pre>	<pre> `ifndef SUPPORT_POWER_DOWN wire G_clock; wire G_sleep = ~( reset   ctrl1   (~ctrl2) ); wire G_chk_en = G_sleep; GATED_OR GATED_G ( .CLOCK( clock ), .SLEEP_CTRL( 1'b0 ),           // bypass clock .CLOCK_GATED(G_clock) ); `else wire G_clock = clock; G_sleep = 0 G_chk_en = 0 `endif Data bypass !! always@(posedge G_clock) begin // gated data if (!G_chk_en) begin if (reset) begin end else if (ctrl1) begin end else if (~ctrl2) begin end end end end </pre>
---	--

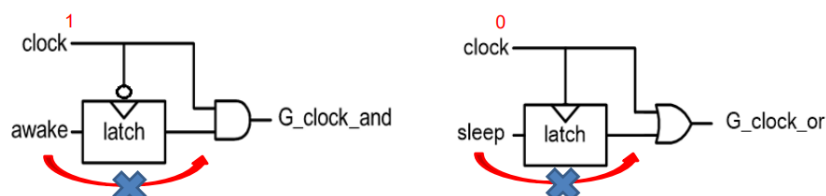
- Clock gating can be implemented by **either AND-gating or OR-gating**
- The control signal can only change in specific half cycle
 - For clock rising edge trigger registers design
 - For AND gate, the allow switching cycle is clock at low period.
 - For OR gate, the allow switching cycle is clock at high period.

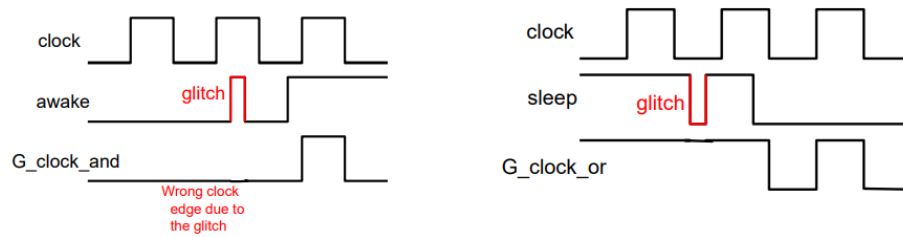


Glitch:



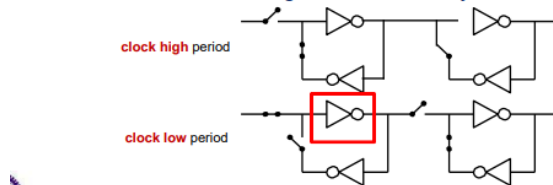
The method of avoid glitch



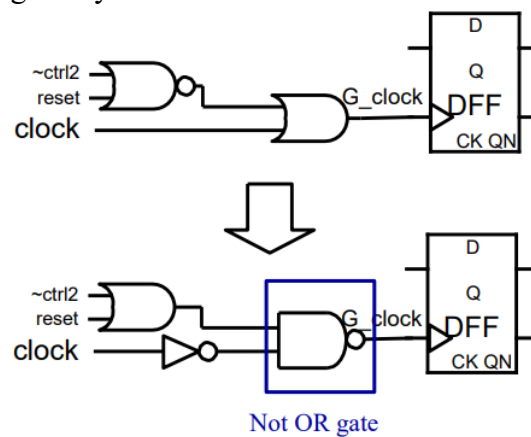


OR-gating is better

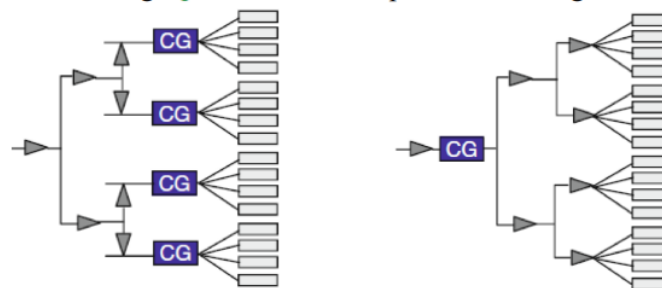
- **OR-gating consumes less power than AND-gating**
 - For OR-gating, the gated clock is tied at high when the register is turned off.
 - No matter data is toggling, the first latch circuits will not be toggled.
 - For AND-gating, the gated clock is tied at low when the register is turned off.
 - The **first latch circuits** will consume power as data input is switching.
- The gating control signals should be generated from clock rising edge
- flip-flops → stable gating the clock as clock high period
 - Consistent with original clock rising edge trigger registers design
 - It is easier for timing control and analysis



Writing OR gate in a single module can prevent the OR gate from being optimized with other logics during the synthesis.



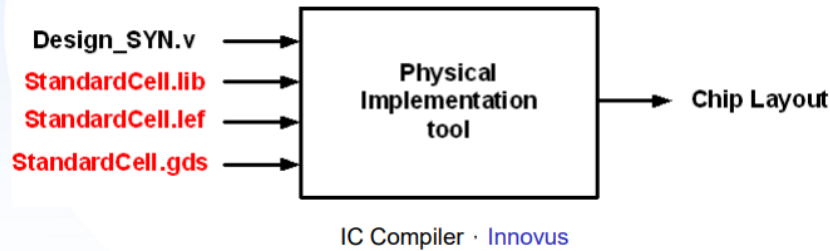
- Power Saving
- Simpler skew management, less area.



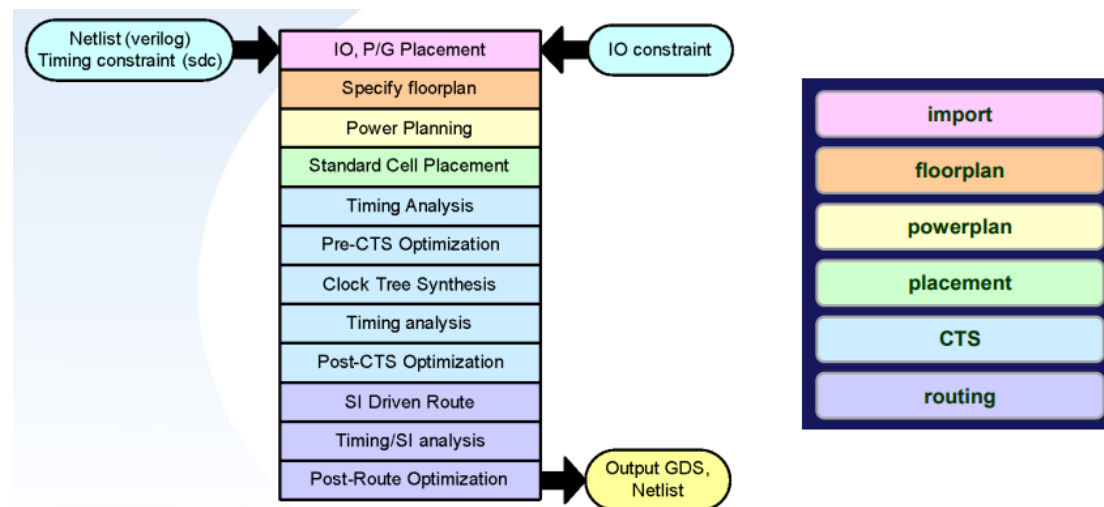
- Keep the gates close to the registers. This allows for a fine-grain control on what to turn off and when. It comes at the expense of a more complex skew control and extra area.
- Move the gating devices higher up in the tree, which has the added advantage that the clock distribution network of the sub-tree is turned off. Modules cannot be turned off as often.

Lecture 11 Cell-Based APR Design Flow

Physical Implementation



APR design flow:



Libraries:

LIB Library : timing information

- Calculate cell delay according to different combinations of input slew rate and output loading capacitance
- Using SignalStorm Library Characterizer

LEF Library : process and cell information

- Metal layer information, routing pitch, default direction
- Cell size, pin position, pin metal layer

RC Extraction

- RC extraction for further power analysis, simulation etc

CelltIC Library

- cdB model (noise model for each cell which will be use in SI optimization phase)

GDSII

- planar geometric shapes and other information about the layout in hierarchical form

When calculating core size of standard cells, the core utilization must be decided first. Usually the core utilization is higher than **70%**

The core size can be calculated as follows

$$\text{Core Size of Standard Cell} = \frac{\text{standard cell area}}{\text{core utilization}}$$

The recommended core shape is a square, i.e. Core Aspect Ratio = 1. Hence the width and height can be calculated as

$$\text{Width} = \text{Height} = \sqrt{\text{Core Size of Standard Cell}}$$

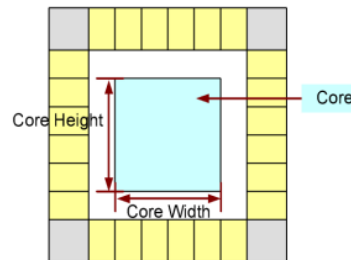
- Note: because stripes and macros will be added, width and height are usually set larger than the value calculated above

E.g.

- Standard cell area = 2,000,000
- Core utilization demanded = 85%
- No macros

$$\text{Core Size of Standard Cell} = \frac{2,000,000}{0.85} = 2,352,941$$

$$\text{Width} = \text{Height} = \sqrt{2,352,941} = 1534$$

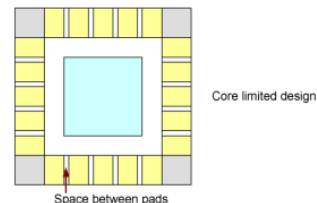
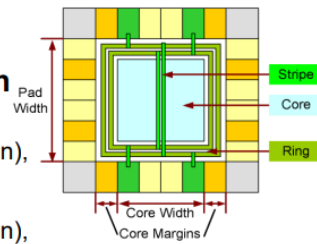


Core margins

- When setup the floorplan, remember to leave enough space for Power/Ground (P/G) ring
 - Note: the width needed for P/G ring will be discussed in power planning

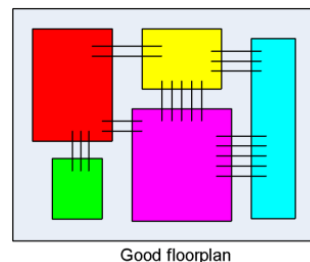
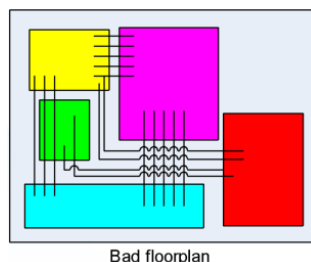
Core limited design or Pad limited design

- Die size determination
 - When pad width $>$ (core width + core margin), die size is decided by **pads**. And it is called pad limited design
 - When pad width $<$ (core width + core margin), die size is decided by **core**. And it is called core limited design
- Adding pad filler
 - Provide power to pad
 - There should be no spacing between pads. Pad filler is necessary for **core limited design**



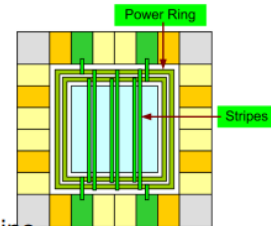
Floorplan Purposes

- Develop early physical layout to ensure design objective can be achieved
 - Minimum area for low cost
 - Minimum congestion for design routable
 - Estimate parasitic for delay calculation
 - Analysis power for reliability
- Gain early visibility into implementation issues
- Different floorplan, different performance



Power issue

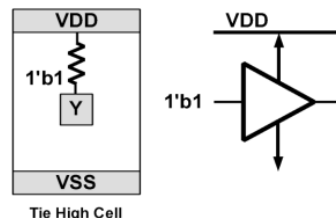
- Metal migration (also known as electro-migration)
 - Under **high currents**, electron collisions with metal grains cause the metal to move. The metal wire may be **open circuit** or **short circuit**.
 - Prevention: sizing power supply lines to ensure that the chip does not fail
 - Experience: make current density of power ring < **1mA/μm**
- IR drop
 - IR drop is the problem of voltage drop of the power and ground due to high current flowing through the power-ground resistive network
 - When there are excessive voltage drops in the power network or voltage rises in the ground network, the device will run at **slower speed**
 - IR drop can cause the chip to fail due to
 - ◆ Performance (circuit running slower than specification)
 - ◆ Functionality problem (setup or hold violations)
 - ◆ Unreliable operation (less noise margin)
 - ◆ Power consumption (leakage power)
 - ◆ Latch up
 - Prevention: adding stripes to avoid IR drop on cell's power line



Connect/Define Global Nets

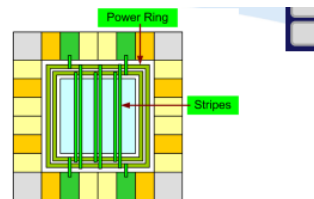
Complete the form

- Add power nets and pins connection list
- Add ground nets and pins connection list
- Tie High: Connect all the 1'b1 to VDD
- Tie Low: Connect all the 1'b0 to GND



Calculate power/ground ring width

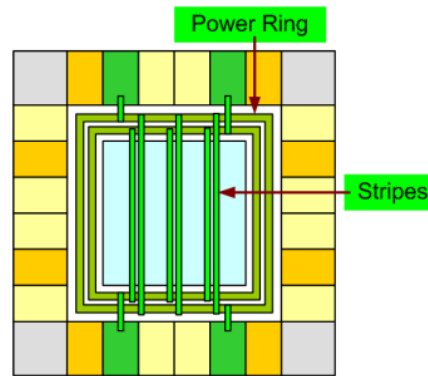
- Experience
 - Gate count = 70 k
 - 4000 Flip-Flops
 - 80% FF with dynamic gated clock
 - Current needed = **0.2mA/MHz**
 - ◆ Note: the value should multiply with **1.8~2 for no gated design**
- Example:
 - Gate count = 200 k
 - No gated clock
 - Clock frequency = 20 MHz
 - Current needed = $(200k/70k) * 0.2 \text{ mA/MHz} * 20\text{MHz} * 2 = 22.86 \text{ mA}$
 - Current density < 1mA/μm
 - The Width of P/G Ring > 22.86 μm
 - In order to avoid the slot rule of wide metal, the largest width is 20 μm (process dependent)
 - Use two set P/G ring for this case



	Without Wire group	With Wire group
Without Interleaving	Without wire groups	With wire groups
Interleaving	Without wire groups	With wire groups

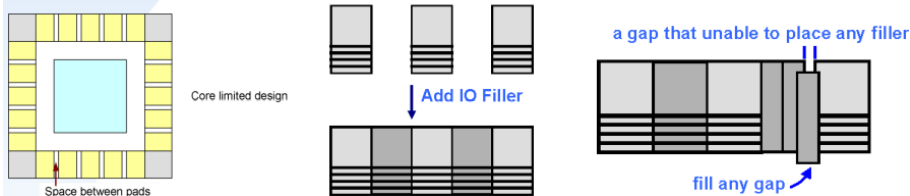
Calculate stripe set

- Experience
 - Add one stripe set per 100 μm
- Example
 - Core width = height = 1600
 - Stripe set added = 15



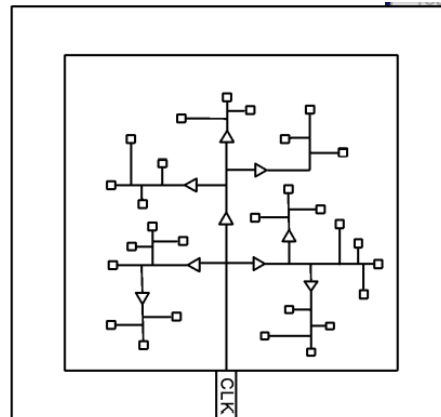
✓ Adding pad filler

- There should be no spacing between pads.
Therefore adding pad filler is necessary for core limited design
- PAD filler must be added before detail route, or there may have some DRC/LVS violations after PAD filler insertion



Clock problem

- Heavy clock net loading
- Long clock insertion delay
- Clock skew
- Skew across clocks
- Clock to signal coupling effect
- Clock is power hungry
- Electromigration on clock net



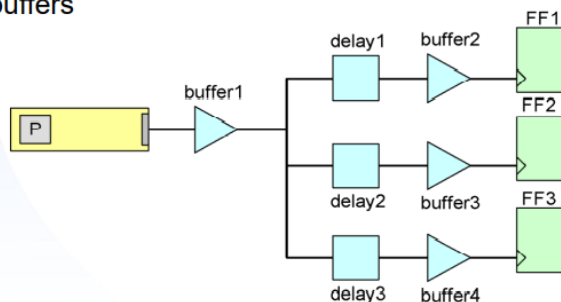
Clock is one of the most important treasure in a chip, do not take it as other use.

The goal of clock tree synthesis includes

- Creating clock tree spec file
- Building a buffer distribution network
- Routing clock nets using CTS-NanoRoute

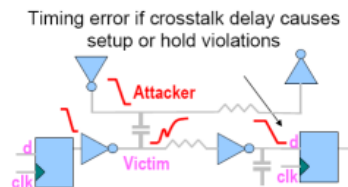
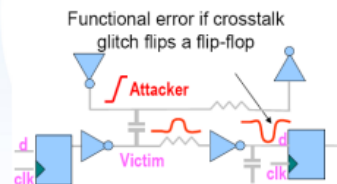
In automatic CTS mode, Innovus will do the following things

- Build the clock buffer tree according to the clock tree specification file
- Balance the clock phase delay with appropriately sized, inserted clock buffers



Signal Integrity (SI) Issue

- Crosstalk
- Charge sharing
- Supply noise
- Leakage
- Propagated noise
- Overshoot
- Under shoot



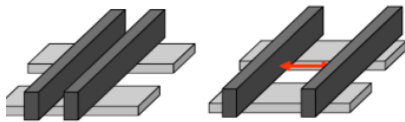
SI closure

- Occur when a design is free from SI induced functional glitch and timing failure

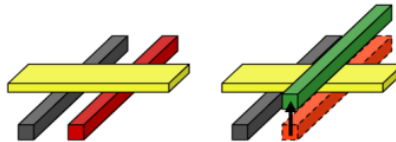
SI prevention (cont.)

- Routing-based SI prevention

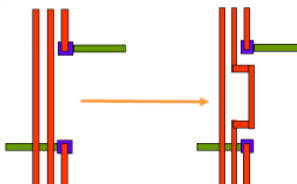
• Wiring Spacing



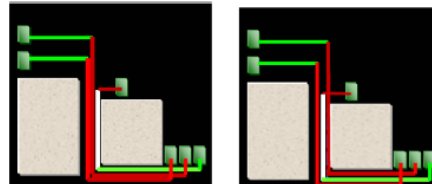
• Layer Switching



• Parallel Wires Reducing



• Net Re-ordering



Antenna Effect

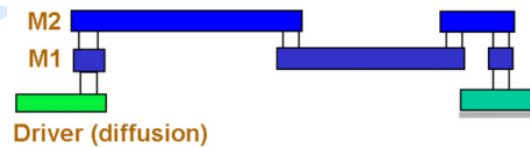
- In a chip manufacturing process, metal is initially deposited so it covers the entire chip
- Then, the unneeded portions of the metal are removed by etching, typically in plasma (charged particles).
- The exposed metal collect charge from plasma and form voltage potential.
- If the voltage potential across the gate oxide becomes large enough, the current can damage the gate oxide.

$$\text{Antenna Ratio} = \frac{\text{Area of process antennas on a node}}{\text{Area of gates to the node}}$$

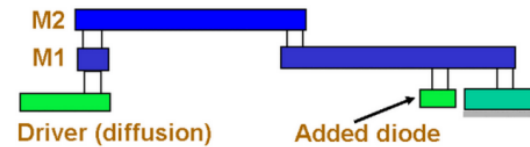
Problem Repair

- Add jumper (changer metal layer)
- Add antenna cell (diode)
- Add buffer

Add jumper



Add diode



If after fixing antenna violation, there still exist violations

- Larger chip area
- Place module precisely

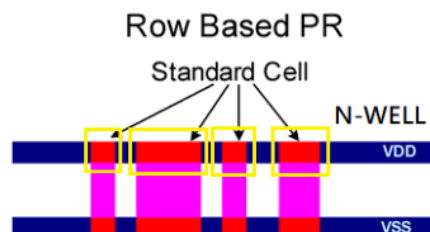
NanoRoute

- Routing the design without creating DRC or LVS violations.
- Routing the design without degrading timing or creating signal integrity violation.

Purpose of adding filler cells

- Fill all the gaps between standard cell instances
- Provide decoupling capacitances to complete connections in the standard cell rows

Metal filler inserted after routing, but before parasitic extraction and GDSII out



LVS v.s. DRC v.s. DRV

LVS: layout verse schematic

- The signals are connect correctly

DRC: design rule check

- Metal density
- Size of poly, metals, vias

DRV: design rule violation

- Fan-out
- Load
- Driving capability
- Wire length

The following files will be generated after APR design flow

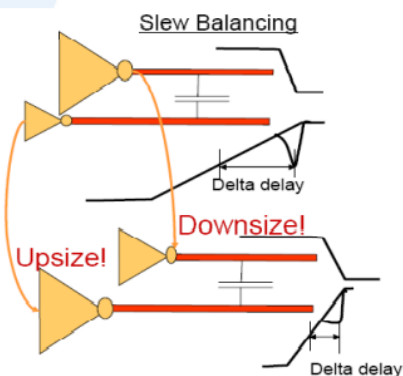
- **CHIP.v**
 - Netlist file for post-layout gate-level simulation
- **CHIP.sdf**
 - Design timing file for post-layout gate-level simulation
- **CHIP.def**
 - Design exchange format relevant to physical layout
- **CHIP.gds**
 - GDSII stream file for Calibre-DRC (Design Rule Check)

Crosstalk prevention – Placement solution

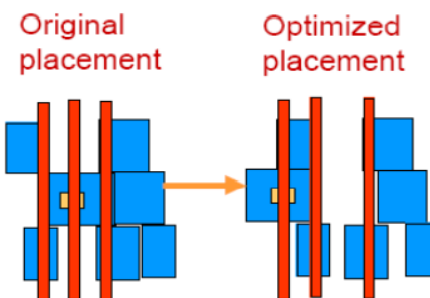
- Insert buffer in lines
- Upsize driver
- Congestion optimization

Crosstalk prevention – Routing solution

- Limit length of parallel nets
- Wider routing grid
- Shield special nets



- Reduce coupling capacitance



Lecture 12 Things to do after layout

Dynamic Power

✓ Switching Power

- $P = 0.5 * C_L V^2 F * A$,
- C_L : output capacitance loading, V: voltage, F: frequency
- A: average switching activity from VCD or computed
- Charge and discharge the loading capacitive

✓ Internal Power (Short circuit power)

- Both NMOS and PMOS are turned on
- Calculated using *.lib file

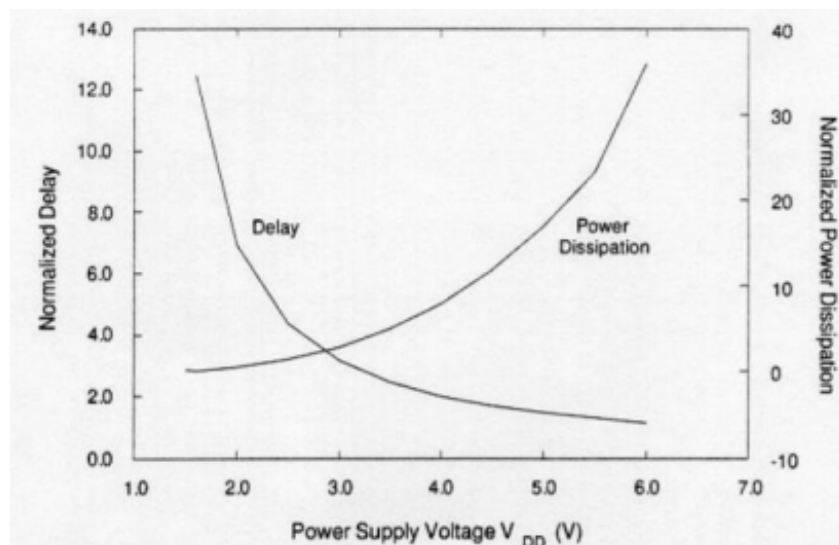
Static Power

✓ Leakage Power

- Power consumed by device when no switching
- Calculated using *.lib file

IR drop (IRD)

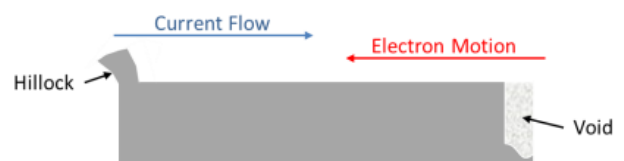
- IR drop is the problem of voltage drop of the power and ground due to high current flowing through the power-ground resistive network.
- The drop in supply voltage over the length of the supply line.
- When there are excessive voltage drops in the power network or voltage rises in the ground network, the device will run at slower speed
- As the process becoming more and more advanced, the IR drop turns to be an important issue.
- IR drop can cause the chip to fail due to
 - ◆ Performance (circuit running slower than specification)
 - ◆ Functionality problem (setup or hold violations)
 - ◆ Unreliable operation (less noise margin)



- Prevention:
 - 1. Adding stripes to avoid IR drop on cell's power line
 - 2. Change the position of power pad to avoid long power line
 - 3. Use square-shaped Chip to avoid long power line
 - 4. Add decoupling cell

Electro -Migration (EM)

- EM is the unwanted transport of material due to movement of ions in a conductor, caused by a transfer of momentum from electrons to these ions.
- High-density current in a narrow metal wire may destroy the wire.
- Performance Degradation
 - Increase/decrease in wire RC
- Can result in catastrophic failure
 - Open
 - Short
- EM analysis solution
 - Calculates the current on each wire and compares it to foundry EM rules.



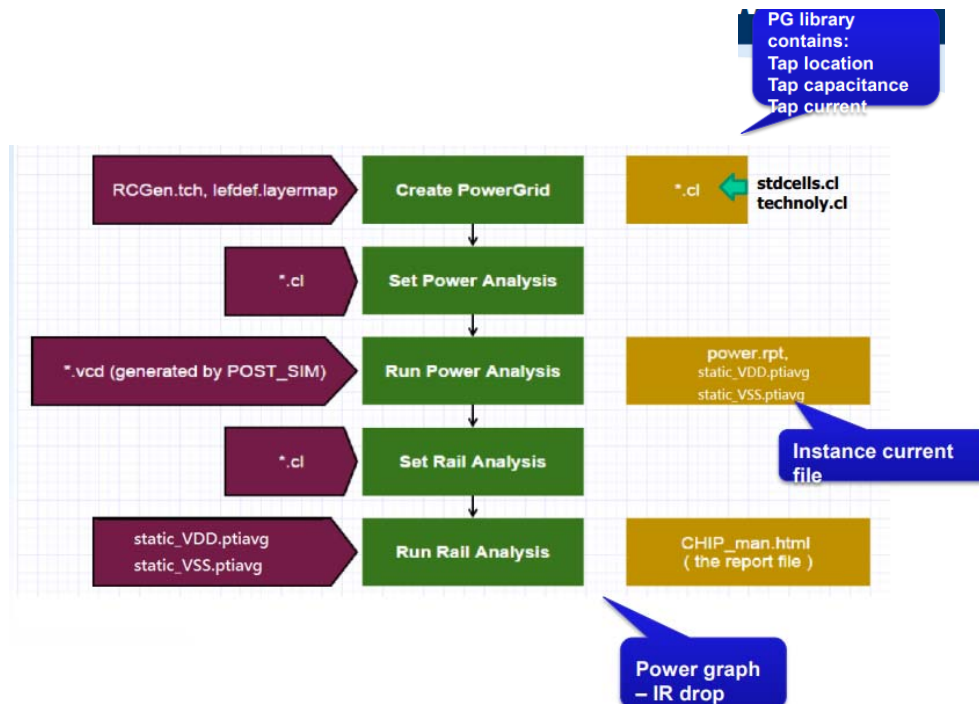
Vector: simulation waveform result for all gates

Vector-based

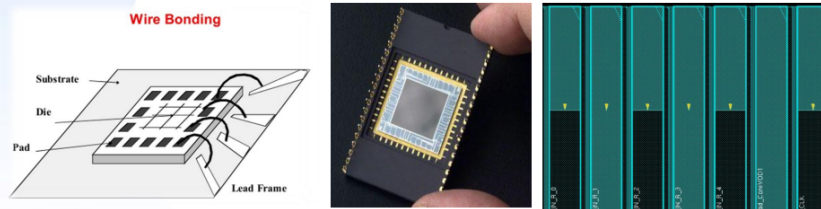
- Uses VCD (waveform file)
 - VCD is waveform file, like fsdb
- Requires:
 - Simulation is possible at the full-chip level
 - Simulation provides sufficient functional coverage of design
 - Vectors include those cause highest power consumption

Vector-independent

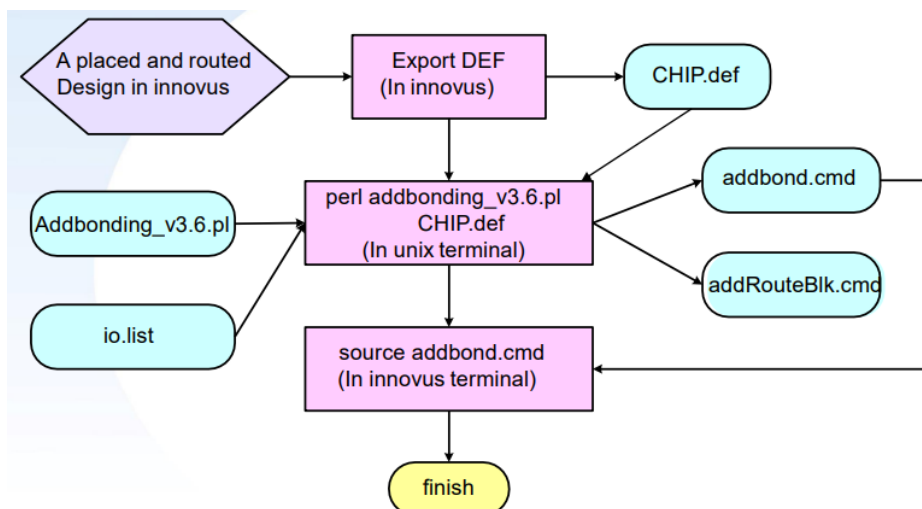
- Uses TWF (timing window file, generated by Static Timing Analysis)
 - Clock domain, skews, slack, arrival times of each pin.
- Requires:
 - A good estimation of input switching frequency



- A bonding pad is used to connect the circuit on a die to a pin on a packaged chip
- Typically, the bonding pad is made from all the metal layers stacked on top of each other and connected through vias



Bonding pads insertion flow:



Introduction of LEF Files

What is lef ?

- Library Exchange Format (LEF)
- LEF defines the elements of an IC process technology and associated library of cell models.
- LEF file can be divide into two part: technology and physical macros

Technology

- A technology LEF file contains all of the LEF technology information for a design, such as placement and routing design rules, and process information for layers.

Physical Macros

- Contains the macro and standard cell information for a design.

Technology LEF file:

Process Technology

- Layers
 - Poly
 - Metal
 - Via
- Design rule
 - Net width
 - Net spacing
 - Area
- Parasitic
 - Resistance
 - Capacitance

APR Technology

- Site
- Routing pitch
- Default direction
- Via rule

Cell library LEF file:

Define physical data for

- Standard cells
- I/O pads
- Memories
- Other hard macros

Describe abstract shape

- A cell library LEF file can include any of the statements as shown in right hand side

Introduction of LIB Files

✓ **The units, operation conditions and template table are defined in the begin of LIB file**

```
library (NangateOpenCellLibrary) {  
  /* Documentation Attributes */  
  date          : "Thu 10 Feb 2011, 18:11:58";  
  revision      : "revision 1.0";  
  comment       : "Copyright (c) 2004-2011 Nangate Inc. All Rights Reserved."  
  
  /* General Attributes */  
  technology    : (cmos);  
  delay_model   : table_lookup;  
  in_place_swap_mode : match_footprints;  
  library_features : {report_delay_calculation,report_power_calculation};  
  
  /* Units Attributes */  
  time_unit      : "ns";  
  leakage_power_unit : "mW";  
  voltage_unit    : "V";  
  current_unit    : "mA";  
  pulling_resistance_unit : "kohm";  
  capacitive_load_unit : (1,ff);  
  
  /* Operation Conditions */  
  nom_process    : 1.00;  
  nom_temperature : 125.00;  
  nom_voltage     : 0.95;  
  
  voltage_map (VDD,0.95);  
  voltage_map (VSS,0.00);  
  
  defineprocess_corner (operating_conditions, string);  
  operating_conditions (slow) {  
    process_corner : "SlowSlow";  
    process        : 1.00;  
    voltage         : 0.95;  
    temperature     : 125.00;  
    tree_type      : balanced_tree;  
  }  
  default_operating_conditions : slow;  
}
```

```
lu_table_template (Timing_7_7) {  
  variable_1 : input_net_transition;  
  variable_2 : total_output_net_capacitance;  
  index_1 ("0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070");  
  index_2 ("0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070");  
}
```

LIB file contains

- Cell area
- Cell Leakage power
- Rise and fall capacitance of pins
- Timing information
- Internal power

The timing information in LIB files

- Cell rise and cell fall
- Rise transition and fall transition

```
lu_table_template (Timing_7_7) {  
    variable_1 : input_net_transition;  
    variable_2 : total_output_net_capacitance;  
    index_1 (*0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070*);  
    index_2 (*0.0010,0.0020,0.0030,0.0040,0.0050,0.0060,0.0070*);  
}
```

The internal power information in LIB files

- Defined at each input pin
- Fall power and rise power

Cell-based Design Flow:

